

Theorem 7.7.14 *After j steps of the conjugate-gradient algorithm (with $r^{(k)} \neq 0$ at each step) we have*

$$\text{span}\{p^{(0)}, \dots, p^{(j)}\} = \text{span}\{r^{(0)}, \dots, r^{(j)}\} = \mathcal{K}_{j+1}(A, r^{(0)}).$$

Proof. The proof is by induction on j . The theorem is trivially true for $j = 0$, since $p^{(0)} = r^{(0)}$. Now assume that the spaces are equal for $j = k$, and we will show that they are also equal for $j = k + 1$. Our first step is to show that

$$\text{span}\{r^{(0)}, \dots, r^{(k+1)}\} \subseteq \mathcal{K}_{k+2}(A, r^{(0)}). \quad (7.7.15)$$

In light of the induction hypothesis, it suffices to show that $r^{(k+1)} \in \mathcal{K}_{k+2}(A, r^{(0)})$. Recalling the recurrence $r^{(k+1)} = r^{(k)} - \alpha_k A p^{(k)}$, we check the status of $A p^{(k)}$. By assumption $p^{(k)} \in \mathcal{K}_{k+1}(A, r^{(0)}) = \text{span}\{r^{(0)}, \dots, A^k r^{(0)}\}$, so

$$A p^{(k)} \in \text{span}\{A r^{(0)}, \dots, A^{k+1} r^{(0)}\} \subseteq \mathcal{K}_{k+2}(A, r^{(0)}).$$

Furthermore $r^{(k)} \in \mathcal{K}_{k+1}(A, r^{(0)}) \subseteq \mathcal{K}_{k+2}(A, r^{(0)})$, so

$$r^{(k+1)} = r^{(k)} - \alpha_k A p^{(k)} \in \mathcal{K}_{k+2}(A, r^{(0)}).$$

This establishes (7.7.15).

Our next step is to establish

$$\text{span}\{p^{(0)}, \dots, p^{(k+1)}\} \subseteq \text{span}\{r^{(0)}, \dots, r^{(k+1)}\}. \quad (7.7.16)$$

By the induction hypothesis,

$$p^{(i)} \in \text{span}\{r^{(0)}, \dots, r^{(k)}\} \quad \text{for } i = 0, \dots, k, \quad (7.7.17)$$

so it suffices to show that $p^{(k+1)} \in \text{span}\{r^{(0)}, \dots, r^{(k+1)}\}$. But this follows immediately from (7.7.7), using (7.7.17) once again.

Putting (7.7.16) and (7.7.15) together, we see that the three subspaces of interest are nested. We can show that they are equal by demonstrating that they all have the same dimension. Since $\mathcal{K}_{k+2}(A, r^{(0)})$ is spanned by a set of $k + 2$ vectors, its dimension is at most $k + 2$. If we can show that the dimension of $\text{span}\{p^{(0)}, \dots, p^{(k+1)}\}$ is exactly $k + 2$, it will follow that all three spaces have dimension $k + 2$ and are therefore equal. But we already know that $p^{(0)}, \dots, p^{(k+1)}$ are nonzero and orthogonal with respect to the energy inner product. Therefore they are linearly independent; they form a basis for $\text{span}\{p^{(0)}, \dots, p^{(k+1)}\}$, whose dimension is therefore $k + 2$. $\square$

The next result is an immediate consequence of Theorems 7.7.6 and 7.7.14.

Corollary 7.7.18 *The error after j steps of the CG algorithm satisfies*

$$e^{(j)} \perp_A \mathcal{K}_j(A, r^{(0)}), \quad j = 1, 2, 3, \dots$$

The search directions $p^{(0)}, \dots, p^{(k)}$ form an energy-orthogonal basis of the Krylov subspace $\mathcal{K}_{k+1}(A, r^{(0)})$. The residuals $r^{(0)}, \dots, r^{(k)}$ also form a basis of the Krylov subspace, and they also have an orthogonality property.

Corollary 7.7.19 *The residuals $r^{(0)}, r^{(1)}, r^{(2)}, \dots$ produced by the CG algorithm are orthogonal with respect to the standard inner product. Hence*

$$r^{(j)} \perp \mathcal{K}_j(A, r^{(0)}), \quad j = 1, 2, 3, \dots$$

Proof. We need to establish that $r^{(k+1)} \perp r^{(0)}, \dots, r^{(k)}$ for all k . This follows immediately from (7.7.10), since $\text{span}\{p^{(0)}, \dots, p^{(k)}\} = \text{span}\{r^{(0)}, \dots, r^{(k)}\}$. $\square$

Now we are in a position to prove that CG can be implemented economically.

Theorem 7.7.20 *For $i = 0, \dots, k-1$, the coefficients c_{ki} defined by (7.7.8) are zero.*

Proof. Referring to (7.7.8), we see that we need to show that $\langle r^{(k+1)}, p^{(i)} \rangle_A = 0$ for $i = 0, \dots, k-1$. Since $\langle r^{(k+1)}, p^{(i)} \rangle_A = \langle r^{(k+1)}, Ap^{(i)} \rangle$, it suffices to show that $r^{(k+1)} \perp Ap^{(i)}$ for $i < k$. We can do this by applying Theorem 7.7.14 twice. We have $p^{(i)} \in \text{span}\{p^{(0)}, \dots, p^{(i)}\} = \text{span}\{r^{(0)}, \dots, A^i r^{(0)}\}$, so $Ap^{(i)} \in \text{span}\{Ar^{(0)}, \dots, A^{i+1}r^{(0)}\} \subseteq \mathcal{K}_{i+2}(A, r^{(0)}) = \text{span}\{r^{(0)}, \dots, r^{(i+1)}\}$. Since $r^{(k+1)}$ is orthogonal to $r^{(0)}, \dots, r^{(i+1)}$ if $i < k$, we conclude that $r^{(k+1)} \perp Ap^{(i)}$ if $i < k$. $\square$

Thanks to Theorem 7.7.20, (7.7.7) reduces to the much simpler expression

$$p^{(k+1)} = r^{(k+1)} + \beta_k p^{(k)}, \quad (7.7.21)$$

where $\beta_k = -c_{kk}$. This shows that $p^{(k+1)}$ can be calculated inexpensively and that the cost does not grow with k . It shows furthermore that there is no need to save the old search directions $p^{(0)}, \dots, p^{(k-1)}$, so the storage requirement does not grow with k . This is a remarkable fact. We embarked on the derivation of the CG algorithm with the idea that we would improve on the method of steepest descent by making use of information from previous steps. We found that by doing so we could minimize the energy norm of the error over larger and larger subspaces. But now (7.7.21) demonstrates that we can accomplish this without having to save much. All the history we need is embodied in $p^{(k)}$.

Theorem 7.7.20 relies heavily on the symmetry property $A = A^T$. The importance of symmetry is easily overlooked, because we have buried A in an inner product. If A were not symmetric, the energy inner product would not be an inner product, since we would not have $\langle v, w \rangle_A = \langle w, v \rangle_A$. If you worked through all the details of this section, you undoubtedly noticed that we used this property from time to time, sometimes in the form $\langle Av, w \rangle = \langle v, Aw \rangle$. The ability to group A with either vector in the inner product is crucial to the development. For example, in the proof of Theorem 7.7.20 we used the identity $\langle r^{(k+1)}, p^{(i)} \rangle_A = \langle r^{(k+1)}, Ap^{(i)} \rangle$.

With one more refinement, our development is complete.

Proposition 7.7.22 *The coefficient β_k in (7.7.21) is given by*

$$\beta_k = \frac{\langle r^{(k+1)}, r^{(k+1)} \rangle}{\langle r^{(k)}, r^{(k)} \rangle}.$$

Exercise 7.7.23 Recalling that $\beta_k = -c_{kk}$, use (7.7.8), (7.4.11), Theorem 7.4.5, and (7.7.11) to prove Proposition 7.7.22. $\square$

We summarize our development as a prototype CG algorithm.

Prototype Conjugate-Gradient Algorithm

$$\begin{aligned} x^{(0)} &\leftarrow \text{initial guess} \\ r^{(0)} &\leftarrow b - Ax^{(0)} \\ p^{(0)} &\leftarrow r^{(0)} \\ \text{for } k = 0, 1, 2, \dots \\ &\left[\begin{array}{l} \alpha_k \leftarrow \langle r^{(k)}, r^{(k)} \rangle / \langle Ap^{(k)}, p^{(k)} \rangle \\ x^{(k+1)} \leftarrow x^{(k)} + \alpha_k p^{(k)} \\ r^{(k+1)} \leftarrow r^{(k)} - \alpha_k Ap^{(k)} \\ \beta_k \leftarrow \langle r^{(k+1)}, r^{(k+1)} \rangle / \langle r^{(k)}, r^{(k)} \rangle \\ p^{(k+1)} \leftarrow r^{(k+1)} + \beta_k p^{(k)} \end{array} \right. \end{aligned} \quad (7.7.24)$$

Thanks to (7.7.11) and Proposition 7.7.22, $\langle r^{(k)}, r^{(k)} \rangle$ appears three times in the loop. Once it has been computed, it gets used over and over. It can even be used in the convergence criterion.

It is a simple matter to translate Algorithm 7.7.24 into Algorithm 7.6.1.

Exercise 7.7.25 Derive Algorithm 7.6.1 from Algorithm 7.7.24. $\square$

Relationship with the Symmetric Lanczos Process

Theorem 7.7.14 shows that the residuals of the CG process form orthogonal bases for Krylov subspaces: $\text{span}\{r^{(0)}, \dots, r^{(j-1)}\} = \mathcal{K}_j(A, r^{(0)})$, $j = 1, 2, 3, \dots$. This suggests a connection between the CG algorithm and the Arnoldi process, which also generates orthogonal bases for Krylov subspaces. Recall that when A is real and symmetric, the Arnoldi process is called the *symmetric Lanczos process*. If we start the symmetric Lanczos process with q_1 equal to a multiple of $r^{(0)}$, then the vectors $q_1, q_2, q_3, \dots$ that it produces will be proportional to $r^{(0)}, r^{(1)}, r^{(2)}, \dots$, since an orthogonal basis for a sequence of nested spaces is uniquely determined up to scalar multiples. (This is essentially the same as the uniqueness of QR decompositions.) Thus the CG and Lanczos processes are producing essentially the same quantities. The connection between CG and symmetric Lanczos is explored further in Exercise 7.7.28. Because of this connection, it is possible to obtain information about the spectrum of A while running the CG algorithm. Of course, if a preconditioner is used, the information is about the eigenvalues of the transformed matrix $\tilde{A}$, not A .

Additional Exercises

Exercise 7.7.26 Prove Theorem 7.7.3. In other words, show that the proof of Theorem 3.5.15 remains valid in the more general context. $\square$

Exercise 7.7.27 Consider the step $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$ by an exact line search. Show that $\alpha_k p^{(k)}$ is the best approximation to $e^{(k)}$ from the one-dimensional space spanned by $p^{(k)}$. Then apply Theorem 7.7.3 to conclude that $e^{(k+1)} \perp_A p^{(k)}$. Thus Proposition 7.7.5 is a special case of Theorem 7.7.3. $\square$

Exercise 7.7.28 In this exercise you will take a look at the relationships between the CG and symmetric Lanczos algorithms. Referring to the prototype CG algorithm (7.7.24), the update $x^{(k+1)} \leftarrow x^{(k)} + \alpha_k p^{(k)}$ is of great interest to us since it brings us closer and closer to the true solution. Notice, however, that it is not essential to the functioning of the algorithm. Since $x^{(k)}$ is not used in the computation of the coefficients α_k and β_k , we could remove that line from the code, and it would still function perfectly, producing the sequences of $p^{(k)}$ and $r^{(k)}$ vectors. Thus the essential updates in (7.7.24) are $r^{(k+1)} = r^{(k)} + \alpha_k A p^{(k)}$ and $p^{(k+1)} = r^{(k+1)} + \beta_k p^{(k)}$. This is a pair of two-term recurrences. In contrast, the Lanczos algorithm runs a single three-term recurrence. In what follows we will view the vectors $p^{(k)}$ and $r^{(k)}$ as columns of matrices, so let us modify the notation, writing p_{k+1} and r_{k+1} in place of $p^{(k)}$ and $r^{(k)}$. Define $R_m = [r_1 \cdots r_m]$, and define P_m analogously.

- Let $q_1, q_2, q_3, \dots$ be the orthonormal vectors produced by the symmetric Lanczos process, starting with q_1 , a multiple of $r_1 = r^{(0)}$. Let $Q_m = [q_1 \cdots q_m]$. Show that $Q_m = R_m D_m$, where D_m is a nonsingular diagonal matrix, assuming all r_j are nonzero.
- Show that the recurrences $p^{(k+1)} = r^{(k+1)} + \beta_k p^{(k)}$, $k = 0, \dots, m-2$, together with $p^{(0)} = r^{(0)}$, can be rewritten as a matrix equation $Q_m = P_m U_m$, where U_m is a bidiagonal, upper-triangular matrix.
- Show that the recurrences $r^{(k+1)} = r^{(k)} + \alpha_k A p^{(k)}$, $k = 0, \dots, m-1$, can be rewritten as a matrix equation $A P_m = Q_m L_m + q_{m+1} \gamma_m e_m^T$, where L_m is bidiagonal and lower triangular, and γ_m is a scalar.
- Multiply the matrix equation from part (c) by U_m on the right to obtain the matrix equation

$$A Q_m = Q_m T_m + q_{m+1} \tilde{\gamma}_m e_m^T, \quad (7.7.29)$$

where $T_m = L_m U_m$. Show that T_m is tridiagonal. (The equation $T_m = L_m U_m$ is an LU decomposition of T_m .)

- Show that T_m is symmetric. (Multiply (7.7.29) by Q_m^T on the left and use orthonormality of the columns of Q_m .)

Equation (7.7.29) is exactly the same as (6.3.20), which is the matrix statement of the three-term recurrence of the symmetric Lanczos process. Thus we have obtained

the Lanczos recursion from the CG recursions. Conversely, we can obtain the CG recursions from the Lanczos recursion: Starting from (7.7.29), we can perform an LU decomposition of T_m , introduce vectors P_m by $P_m = Q_m U_m^{-1}$, and arrive at a pair of two-term recurrences equivalent to the CG algorithm. $\square$

7.8 CONVERGENCE OF THE CG ALGORITHM

In this section we explore the convergence properties of the CG algorithm. All of the results given here are stated for the unpreconditioned algorithm. To get analogous results for the preconditioned CG method, just replace A by the transformed matrix $\tilde{A} = R^{-T} A R^{-1}$.

Theorem 7.8.1 *The CG algorithm, applied to an $n \times n$ positive definite system $Ax = b$, arrives at the exact solution in n or fewer steps.*

Proof. If we have taken $n - 1$ steps without getting to x , the nonzero residuals $r^{(0)}, \dots, r^{(n-1)}$ form an orthogonal basis of $\mathbb{R}^n$, by Corollary 7.7.19. After n steps,

$$r^{(n)} \perp r^{(i)} \quad \text{for } i = 0, \dots, n-1.$$

Since $r^{(n)}$ is orthogonal to an entire basis of $\mathbb{R}^n$, we must have $r^{(n)} = 0$, $e^{(n)} = 0$, and $x^{(n)} = x$. $\square$

Exercise 7.8.2 Devise two other proofs of Theorem 7.8.1 based on (a) $e^{(j)} \perp_A p^{(i)}$, (b) minimization of energy norm. $\square$

Theorem 7.8.1 holds for any conjugate-direction method that performs exact line searches. At each step the dimension of the space in which the search is taking place is reduced by one, because each new search direction is conjugate to all of the previous ones.

Theorem 7.8.1 is an interesting property, but it is not of immediate practical importance. Consider, for example, model problem (7.1.8) with $h = 1/40$. The dimension of the coefficient matrix is $39^2 = 1524$, so the CG algorithm (ignoring roundoff errors) is guaranteed to get to the exact solution in 1524 steps or fewer. In fact we would like to get close in a lot fewer steps than that, and Example 7.6.5 shows that we can. In that example, the CG algorithm with SSOR preconditioner was able to get a satisfactory approximation in about 30 iterations.

We will ignore the finite-termination property, by and large, and continue to view CG as an iterative method. Above all, we want to know how quickly the iterates get close to x . The following theorem gives some information along those lines.

Theorem 7.8.3 *The conjugate gradient errors $e^{(j)} = x - x^{(j)}$ satisfy*

$$\|e^{(j)}\|_A \leq 2\|e^{(0)}\|_A \left(\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \right)^j.$$

A proof of Theorem 7.8.3 is outlined in Exercise 7.8.13. It is built on material that is yet to come in this section and the properties of Chebyshev polynomials developed in Exercise 6.3.38.

Theorem 7.8.3, like all rigorous error bounds, tends to be pessimistic, because it is built by stringing together a number of estimates, some of which may be quite crude. Theorem 7.8.1 shows that for $j \geq n$ it is certainly very pessimistic. Of course, we normally expect to stop long before j approaches n .

Even though it is pessimistic, Theorem 7.8.3 does give us useful information about the performance of CG. The convergence rate is no worse than linear, with convergence ratio

$$\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1}.$$

From this ratio it is clear that we can guarantee rapid convergence by making A well conditioned. This reinforces the idea, stated initially for the steepest descent algorithm, that performance can be improved by using a preconditioner that lowers the condition number of the matrix.

Example 7.8.4 Consider the matrix A of the model problem (7.1.8). In Exercise 7.3.37 you showed that $\kappa_2(A) = \cot^2(\pi h/2)$. Thus $\kappa_2(A) \approx (\frac{2}{\pi h})^2$ and

$$\frac{\sqrt{\kappa_2(A)} - 1}{\sqrt{\kappa_2(A)} + 1} \approx \frac{1 - \pi h/2}{1 + \pi h/2} \approx 1 - \pi h.$$

To make a comparison with the SOR method, we refer to Table 7.5, which states that SOR with optimal ω has linear convergence with ratio $\rho(G) \approx 1 - 2\pi h$. This suggests that optimal SOR will converge in about half as many iterations as CG with no preconditioner. However, it is not fair to compare the two convergence rates too closely. The rate for SOR is precise, whereas the rate for CG is a crude upper bound. It would be fairer to say that, at worst, unpreconditioned CG converges at roughly the same rate as optimal SOR. This conclusion is consistent with what we observed in Examples 7.2.21 and 7.6.2.

We can expect CG to do much better if we use a preconditioner that effectively replaces A by a transformed matrix $\tilde{A}$ for which $\kappa_2(\tilde{A}) \ll \kappa_2(A)$. $\square$

Exercise 7.8.5 Use Taylor expansions to verify the approximations made in Example 7.8.4. $\square$

The proof of Theorem 7.8.3 will make use of the fact that CG minimizes the energy norm of the error over subspaces of greater and greater dimension (Theorem 7.7.6). In the remainder of this section, we elaborate on this optimality property and derive some consequences that allow us to prove Theorem 7.8.3 and other useful properties of the CG method.

Theorem 7.8.6 *The iterates of the CG algorithm satisfy*

$$\|x - x^{(j)}\|_A = \min \left\{ \|x - y\|_A \mid y \in x^{(0)} + \mathcal{K}_j(A, r^{(0)}) \right\}.$$

Proof. This is basically a restatement of Theorem 7.7.6. Theorem 7.7.14 shows that the space $\mathcal{S}_j$ of Theorem 7.7.6 is none other than the Krylov subspace $\mathcal{K}_j(A, r^{(0)})$. Given $p \in \mathcal{S}_j$, let $y = x^{(0)} + p \in x^{(0)} + \mathcal{S}_j$. Then, since $x - y = e^{(0)} - p$, we see that minimizing $\|e^{(0)} - p\|_A$ over all p in the subspace $\mathcal{S}_j$ is the same as minimizing $\|x - y\|_A$ over all y in the shifted subspace $x^{(0)} + \mathcal{S}_j$. $\square$

The Krylov subspace $\mathcal{K}_j(A, r^{(0)})$ is the set of all linear combinations of $r^{(0)}, Ar^{(0)}, \dots, A^{j-1}r^{(0)}$. Thus a generic member of $\mathcal{K}_j(A, r^{(0)})$ has the form

$$c_0 r^{(0)} + c_1 A r^{(0)} + \dots + c_{j-1} A^{j-1} r^{(0)} = (c_0 I + c_1 A + \dots + c_{j-1} A^{j-1}) r^{(0)} = q(A) r^{(0)},$$

where $q(z) = c_0 + c_1 z + \dots + c_{j-1} z^{j-1}$ is a polynomial of degree less than j . Conversely, given any polynomial q of degree less than j , the vector $q(A)r^{(0)}$ belongs to $\mathcal{K}_j(A, r^{(0)})$. Let $\mathcal{P}_{j-1}$ denote the space of all polynomials of degree less than j . Then

$$\mathcal{K}_j(A, r^{(0)}) = \left\{ q(A)r^{(0)} \mid q \in \mathcal{P}_{j-1} \right\}.$$

We have noted in Theorem 7.8.6 that $x^{(j)}$ is the optimal member of $x^{(0)} + \mathcal{K}_j(A, r^{(0)})$ in a certain sense. Let z denote an arbitrary element of $x^{(0)} + \mathcal{K}_j(A, r^{(0)})$. Then $z = x^{(0)} + q(A)r^{(0)}$ for some $q \in \mathcal{P}_{j-1}$. Viewing z as an approximation to x , we consider $e = x - z$. Then, recalling that $r^{(0)} = Ae^{(0)}$, we have

$$e = x - x^{(0)} - q(A)r^{(0)} = e^{(0)} - q(A)Ae^{(0)} = (I - q(A)A)e^{(0)} = p(A)e^{(0)},$$

where $p(z) = 1 - zq(z)$. Note that $p \in \mathcal{P}_j$ and $p(0) = 1$. Conversely, for any $p \in \mathcal{P}_j$ satisfying $p(0) = 1$, there exists $q \in \mathcal{P}_{j-1}$ such that $p(z) = 1 - zq(z)$. Thus $e = x - z$ for some $z \in x^{(0)} + \mathcal{K}_j(A, r^{(0)})$ if and only if $e = p(A)e^{(0)}$ for some $p \in \mathcal{P}_j$ satisfying $p(0) = 1$. This observation allows us to rewrite Theorem 7.8.6 as follows.

Theorem 7.8.7 *Let $x^{(j)}$ be the j th iterate of the CG method, and let $e^{(j)} = x - x^{(j)}$. Then*

$$\|e^{(j)}\|_A = \min_{\substack{p \in \mathcal{P}_j \\ p(0)=1}} \|p(A)e^{(0)}\|_A,$$

where $\mathcal{P}_j$ denotes the space of polynomials of degree less than or equal to j .

This theorem enables us to obtain a useful bound on $\|e^{(j)}\|_A$ in terms of the eigenvalues of A . To this end, let $v_1, \dots, v_n$ be a complete orthonormal set of eigenvectors of A . Then $v_1, \dots, v_n$ is a basis of $\mathbb{R}^n$, so

$$e^{(0)} = c_1 v_1 + c_2 v_2 + \dots + c_n v_n,$$

where $c_1, \dots, c_n$ are uniquely determined (but unknown) coefficients. Let $\lambda_1, \dots, \lambda_n$ denote the eigenvalues associated with $v_1, \dots, v_n$, respectively. Then $p(A)v_k = p(\lambda_k)v_k$ for all k , so

$$\|p(A)e^{(0)}\|_A^2 = (p(A)e^{(0)})^T A p(A)e^{(0)}$$

$$\begin{aligned}
&= \sum_{k=1}^n \sum_{l=1}^n c_k c_l p(\lambda_l) \lambda_k p(\lambda_k) v_l^T v_k \\
&= \sum_{k=1}^n c_k^2 \lambda_k p(\lambda_k)^2.
\end{aligned}$$

Combining this expansion with Theorem 7.8.7, we obtain

$$\|e^{(j)}\|_A^2 = \min_{\substack{p \in \mathcal{P}_j \\ p(0)=1}} \sum_{k=1}^n c_k^2 \lambda_k p(\lambda_k)^2. \quad (7.8.8)$$

To get a result that does not make reference to the unknown coefficients $c_1, \dots, c_n$, notice that

$$\|e^{(0)}\|_A^2 = e^{(0)T} A e^{(0)} = \sum_{k=1}^n \lambda_k c_k^2,$$

and therefore

$$\sum_{k=1}^n c_k^2 \lambda_k p(\lambda_k)^2 \leq \max_k p(\lambda_k)^2 \|e^{(0)}\|_A^2.$$

Combining this observation with (7.8.8), we obtain the following theorem.

Theorem 7.8.9 *Let $e^{(j)} = x - x^{(j)}$ be the error after j iterations of the CG algorithm applied to a matrix A with eigenvalues $\lambda_1, \dots, \lambda_n$. Then*

$$\|e^{(j)}\|_A \leq \left(\min_{\substack{p \in \mathcal{P}_j \\ p(0)=1}} \max_{1 \leq k \leq n} |p(\lambda_k)| \right) \|e^{(0)}\|_A,$$

where $\mathcal{P}_j$ denotes the space of polynomials of degree less than $j + 1$.

Theorem 7.8.3 follows from Theorem 7.8.9 by taking p to be an appropriately normalized Chebyshev polynomial. See Exercise 7.8.13.

If we divide both sides of the inequality in Theorem 7.8.9 by $\|e^{(0)}\|_A$, we obtain

$$\frac{\|e^{(j)}\|_A}{\|e^{(0)}\|_A} \leq \min_{\substack{p \in \mathcal{P}_j \\ p(0)=1}} \max_{k=1, \dots, n} |p(\lambda_k)|. \quad (7.8.10)$$

The left-hand side is a ratio that measures how much better $x^{(j)}$ approximates x than $x^{(0)}$ does. In problems for which we have no *a priori* information about the solution, the initial guess $x^{(0)} = 0$ is as good as any. In this case we have $e^{(0)} = x$, and the left-hand side of (7.8.10) becomes $\|x - x^{(j)}\|_A / \|x\|_A$, which is the relative error in the approximation of x by $x^{(j)}$. Thus the relative error is bounded above by a quantity that depends only on the eigenvalues of A . This is the “no information” case. In cases where we have some *a priori* information about x , we can usually choose an $x^{(0)}$ for which $\|e^{(0)}\|_A < \|x\|_A$, thereby obtaining a better result.

Theorem 7.8.9 leads to yet another proof of the finite termination property of CG. Taking $j = n$ we see that for any $p \in \mathcal{P}_n$ with $p(0) = 1$ we have

$$\|e^{(n)}\|_A \leq \left(\max_{1 \leq k \leq n} |p(\lambda_k)| \right) \|e^{(0)}\|_A. \quad (7.8.11)$$

Now let $p(z) = c(z - \lambda_1) \cdots (z - \lambda_n) \in \mathcal{P}_n$, where c is chosen that $p(0) = 1$. (This is possible because all of the λ_j are nonzero.) For this p the right-hand side of the (7.8.11) is zero, so $e^{(n)} = 0$, i.e. $x^{(n)} = x$.

The finite termination property can be sharpened in the following way.

Theorem 7.8.12 *Suppose A has only j distinct eigenvalues. Then the CG algorithm terminates with the exact solution in at most j steps.*

Proof. Let $\mu_1, \dots, \mu_j$ be the distinct eigenvalues of A . Let $p(z) = c(z - \mu_1) \cdots (z - \mu_j)$, where c is chosen so that $p(0) = 1$. Then $p \in \mathcal{P}_j$ and $p(\lambda_k) = 0$ for $k = 1, \dots, n$, so $e^{(j)} = 0$ by Theorem 7.8.9. $\square$

Thus termination will take place quite early if the matrix that has only a few distinct eigenvalues of high multiplicity. More generally, consider a matrix whose eigenvalues are located in j tight clusters, where $j \ll n$. Choose $p(z) = c(z - \mu_1)(z - \mu_2) \cdots (z - \mu_j)$, where $\mu_1, \dots, \mu_j$ are representatives of the j distinct clusters. Since every eigenvalue λ_k is close to one of the μ_m , and every $p(\mu_m)$ is zero, $\max_k |p(\lambda_k)|$ will be small. Therefore, by Theorem 7.8.9, $\|e^{(j)}\|_A$ will be small. We conclude that if A is a matrix whose eigenvalues lie in j tight clusters, then x_j will be close to x .

A preconditioner that makes the condition number small is actually pushing the eigenvalues into a single cluster, as the condition number is just the ratio λ_n/λ_1 . Now our discussion shows that a single cluster is not really needed. A preconditioner that can push the eigenvalues into a few clusters will also be effective. In particular, if the eigenvalues are pushed into one cluster with a few outliers, this is no worse than having a few clusters.

Additional Exercises

Exercise 7.8.13 In this exercise you will prove Theorem 7.8.3. Let the eigenvalues of A be numbered in the order $\lambda_1 \leq \cdots \leq \lambda_n$. From Theorem 7.8.9 we see that we can show that $\|e^{(j)}\|_A$ is small by finding a polynomial $p_j \in \mathcal{P}_j$ for which $p_j(0) = 1$, and $p_j(\lambda_1), p_j(\lambda_2), \dots, p_j(\lambda_n)$ are all small. One way to ensure that all of the $p_j(\lambda_m)$ are small is to make p_j small on the entire interval $[\lambda_1, \lambda_n]$. The best tool for this task is a shifted, scaled Chebyshev polynomial.

- (a) Determine the affine transformation $w = \alpha x + \beta$ that maps $[\lambda_1, \lambda_n]$ onto $[-1, 1]$, with $\lambda_n \rightarrow -1$ and $\lambda_1 \rightarrow 1$. Show that this transformation maps $0 \rightarrow (\lambda_n + \lambda_1)/(\lambda_n - \lambda_1)$.

(b) Let

$$p_j(x) = \frac{T_j\left(\frac{\lambda_n + \lambda_1 - 2x}{\lambda_n - \lambda_1}\right)}{T_j\left(\frac{\lambda_n + \lambda_1}{\lambda_n - \lambda_1}\right)},$$

where T_j is the j th Chebyshev polynomial (Exercise 6.3.38). Then evidently $p_j \in \mathcal{P}_j$ and $p_j(0) = 1$. Show that

$$\max_{x \in [\lambda_1, \lambda_n]} |p_j(x)| = \frac{1}{T_j\left(\frac{\lambda_n + \lambda_1}{\lambda_n - \lambda_1}\right)},$$

and

$$\frac{\lambda_n + \lambda_1}{\lambda_n - \lambda_1} = \frac{\kappa_2(A) + 1}{\kappa_2(A) - 1}.$$

(c) Show that if $t > 0$ and $x = \cosh t > 1$, then $e^t = \cosh t + \sinh t = x + \sqrt{x^2 - 1}$. Show further that if $x = (\kappa_2(A) + 1)/(\kappa_2(A) - 1)$, then $e^t = (\sqrt{\kappa_2(A)} + 1)/(\sqrt{\kappa_2(A)} - 1)$.

(d) In part (f) of Exercise 6.3.38 we showed that if $x > 1$, and $t > 0$ is such that $x = \cosh t$, then $T_j(x) \geq \frac{1}{2}(e^t)^j$ for all j . Use this bound with $x = (\kappa_2(A) + 1)/(\kappa_2(A) - 1)$, together with the identity from part (c), to show that (for this particular x)

$$T_j(x) \geq \frac{1}{2} \left(\frac{\sqrt{\kappa_2(A)} + 1}{\sqrt{\kappa_2(A)} - 1} \right)^j.$$

(e) Put the pieces together to prove Theorem 7.8.3.

□

Exercise 7.8.14 In this exercise you will show that the convergence rate of the CG method is sometimes much better than Theorem 7.8.3 would indicate. Before working this exercise, you need to have worked Exercise 7.8.13. Suppose the spectrum of A consists of a tightly packed kernel of eigenvalues plus a few outliers. Let $\lambda_1, \dots, \lambda_i$ denote the outliers, and suppose the remaining eigenvalues are ordered so that $\lambda_{i+1} \leq \dots \leq \lambda_n$. Define a “reduced condition number” $\hat{\kappa} = \lambda_n/\lambda_{i+1}$. This is the condition number of the operator obtained by restricting A to the invariant subspace associated with the non-outlier eigenvalues. If the furthest outlier is far from the tightly packed core, $\hat{\kappa}$ will be significantly smaller than $\kappa_2(A)$. Let $q_i(x) = c(x - \lambda_1)(x - \lambda_2) \cdots (x - \lambda_i)$, where c is chosen so that $q_i(0) = 1$. For each j define

$$p_{i+j}(x) = q_i(x) \frac{T_j\left(\frac{\lambda_n + \lambda_{i+1} - 2x}{\lambda_n - \lambda_{i+1}}\right)}{T_j\left(\frac{\lambda_n + \lambda_{i+1}}{\lambda_n - \lambda_{i+1}}\right)}.$$

Then evidently $p_{i+j} \in \mathcal{P}_{i+j}$, $p_{i+j}(0) = 1$, $p_{i+j}(\lambda_k) = 0$ for $k = 1, \dots, i$, and $|p_{i+j}|$ is small on $[\lambda_{i+1}, \lambda_n]$. Let $C = \max_{x \in [\lambda_{i+1}, \lambda_n]} |q_i(x)|$. Drawing on techniques from Exercise 7.8.13, prove that

$$\|e^{(i+j)}\|_A \leq 2C \|e^{(0)}\|_A \left(\frac{\sqrt{\hat{\kappa}} - 1}{\sqrt{\hat{\kappa}} + 1} \right)^j.$$

This shows that after the i th iteration, the convergence rate is governed by $\hat{\kappa}$ instead of $\kappa_2(A)$. Thus we have much faster convergence than we would have expected from looking at $\kappa_2(A)$ alone. $\square$

Exercise 7.8.15 Show that the iterates of the preconditioned CG algorithm with preconditioner M satisfy

$$\begin{aligned} \|x - x^{(j)}\|_A &= \min \left\{ \|x - y\|_A \mid y \in x^{(0)} + \mathcal{K}_j(M^{-1}A, s^{(0)}) \right\} \\ &= \min_{\substack{p \in \mathcal{P}_j \\ p(0)=1}} \|p(M^{-1}A)e^{(0)}\|_A, \end{aligned}$$

where $s^{(0)} = M^{-1}r^{(0)}$. $\square$

7.9 KRYLOV SUBSPACE METHODS FOR INDEFINITE AND NONSYMMETRIC PROBLEMS

The CG algorithm is one of a large class of algorithms known as *Krylov subspace methods* because they generate sequences of Krylov subspaces. CG was designed for symmetric, positive-definite matrices. For other types of matrices other Krylov subspace methods have been developed. In this section we will give a brief overview of some of the more popular methods. For more information see [5], [26], [34], or [35].

Let us begin with symmetric, indefinite matrices. First of all, even though the CG algorithm was designed for positive-definite matrices, it is possible to run CG on indefinite problems, often successfully. However, the algorithm can occasionally break down, since the computation of α_k in (7.7.24) requires division by $\langle Ap^{(k)}, p^{(k)} \rangle$, which can be zero if A is indefinite.

It is natural to ask whether there are algorithms for symmetric indefinite matrices that are like CG but do not break down. Fortunately the answer is yes. The symmetric Lanczos process (6.3.23), which generates a sequence of Krylov subspaces via the economical 3-term recurrence

$$q_{k+1}\beta_k = Aq_k - q_k\alpha_k - q_{l-1}\beta_{k-1},$$

works for all symmetric matrices, definite or not.⁵ Thus we can use the Lanczos process to generate the Krylov subspaces.

But now there is a second question. How do we pick an appropriate approximation from the Krylov subspace? CG minimizes $\|e^{(j)}\|_A$, the energy norm of the error. Since $r^{(j)} = Ae^{(j)}$, we have $\|e^{(j)}\|_A = e^{(j)T} Ae^{(j)} = r^{(j)T} A^{-1} r^{(j)} = \|r^{(j)}\|_{A^{-1}}$, so CG is also minimizing the A^{-1} -norm of the residual. In the indefinite case, neither $\|x\|_A$ nor $\|x\|_{A^{-1}}$ is meaningful. The definitions $\|x\|_A = \sqrt{\langle Ax, x \rangle}$ and $\|x\|_{A^{-1}} = \sqrt{\langle A^{-1}x, x \rangle}$ make no sense, since the quantities under the square root signs can be negative. Thus we need another criterion for choosing an approximate solution. The simplest idea is to choose $x^{(j)}$ so that $\|r^{(j)}\|_2$, the Euclidean norm of the residual, is minimized. Just as for CG, the minimization is taken over the translated Krylov subspace $x^{(0)} + \mathcal{K}_j(A, r^{(0)})$. The resulting algorithm is called MINRES (MINimum RESidual) [53].

This procedure can also be carried out for nonsymmetric problems. In this case the Krylov subspaces have to be generated by the Arnoldi process (6.3.6) instead of the symmetric Lanczos process. The algorithm that does this, choosing $x^{(j)}$ to minimize $\|r^{(j)}\|_2$ at each step, is called GMRES (Generalized Minimum RESidual) [60]. The Arnoldi process does not use short recurrences. We see from (6.3.3) that the computation of each new vector requires all of the previous vectors. Thus with each step of the algorithm, storage space for one more vector is needed. The amount of arithmetic per step also increases linearly. These considerations place severe restrictions on the number of GMRES iterations if A is extremely large. Therefore it is a common practice to run GMRES in "restart mode." GMRES(m) is the variant of GMRES that restarts the Arnoldi process every m iterations, using the most recent iterate as the new initial guess. In this mode the storage requirement is held to $O(m)$ vectors. Unfortunately convergence is not guaranteed. It is difficult to know in advance how big m needs to be in order to ensure success. In spite of its shortcomings, GMRES has been very popular.

The algorithms that we have discussed so far choose the new iterate at each step by a minimization criterion. Other algorithms use a criterion based on orthogonalization. Recall that the CG algorithm chooses $x^{(j)}$ so that the error $e^{(j)}$ is orthogonal to $\mathcal{K}_j(A, r^{(0)})$ in the energy inner product, that is, $e^{(j)} \perp_A \mathcal{K}_j(A, r^{(0)})$ (Corollary 7.7.18). This is equivalent to the residual being orthogonal to $\mathcal{K}_j(A, r^{(0)})$ in the standard inner product: $r^{(j)} \perp \mathcal{K}_j(A, r^{(0)})$ (Corollary 7.7.19).

For CG (with positive definite A) the orthogonality property is linked directly to the minimality of $\|e^{(j)}\|_A$ and $\|r^{(j)}\|_{A^{-1}}$ via Theorem 7.7.3, using the energy inner product. In the indefinite and nonsymmetric cases this connection breaks down, since we no longer have the energy inner product at our disposal. This does not stop us

⁵In Exercise 7.7.28 the connection between the Lanczos and CG recursions was discussed. It was observed that to get from the Lanczos to the CG recursion an LU decomposition of the tridiagonal matrix T is needed. In the positive definite case, the LU decomposition always exists, but in the indefinite case it may not. The CG algorithm breaks down in exactly those situations where the LU decomposition (without pivoting) fails to exist.

from building an algorithm that chooses $x^{(j)}$ so that the *Galerkin condition*

$$r^{(j)} \perp \mathcal{K}_j(A, r^{(0)}) \quad (7.9.1)$$

is satisfied. The algorithm SYMMLQ [53] operates on symmetric indefinite matrices, generates Krylov subspaces by the symmetric Lanczos process, and chooses $x^{(j)}$ at each step in such a way that (7.9.1) is satisfied. This criterion does not correspond to the minimization of $r^{(j)}$ in some norm, so SYMMLQ is different from MINRES. The algorithm can break down occasionally, because it can happen that (7.9.1) has either no solution or infinitely many solutions. Breakdown at one step does not preclude moving onto the next step, since the underlying symmetric Lanczos process does not break down. However, MINRES is generally preferred over SYMMLQ.

Another question that arises naturally is this. In the nonsymmetric case, are there Krylov subspace methods that use short recurrences, thereby circumventing the storage and execution time difficulties of GMRES? Perhaps we can build such a method based on an orthogonality criterion like (7.9.1) instead of a minimization criterion. It turns out that we can, but we have to give up on orthogonality and settle for the weaker condition known as biorthogonality. The biconjugate-gradient (BiCG) algorithm is a generalization of the CG algorithm that generates sequences of dual vectors $\tilde{r}^{(0)}, \tilde{r}^{(1)}, \tilde{r}^{(2)}, \dots$ and $\tilde{p}^{(0)}, \tilde{p}^{(1)}, \tilde{p}^{(2)}, \dots$ along with the primal vectors.

Prototype BiCG Algorithm

$$\begin{aligned} x^{(0)} &\leftarrow \text{initial guess} \\ r^{(0)} &\leftarrow b - Ax^{(0)} \\ p^{(0)} &\leftarrow r^{(0)} \\ \tilde{r}^{(0)} &\leftarrow \text{user's choice s.t. } \langle r^{(0)}, \tilde{r}^{(0)} \rangle \neq 0 \\ \tilde{p}^{(0)} &\leftarrow \tilde{r}^{(0)} \\ \text{for } k = 0, 1, 2, \dots \\ &\left[\begin{aligned} \alpha_k &\leftarrow \langle r^{(k)}, \tilde{r}^{(k)} \rangle / \langle Ap^{(k)}, \tilde{p}^{(k)} \rangle \\ x^{(k+1)} &\leftarrow x^{(k)} + \alpha_k p^{(k)} \\ r^{(k+1)} &\leftarrow r^{(k)} - \alpha_k Ap^{(k)} \\ \tilde{r}^{(k+1)} &\leftarrow \tilde{r}^{(k)} - \alpha_k A^T \tilde{p}^{(k)} \\ \beta_k &\leftarrow \langle r^{(k+1)}, \tilde{r}^{(k+1)} \rangle / \langle r^{(k)}, \tilde{r}^{(k)} \rangle \\ p^{(k+1)} &\leftarrow r^{(k+1)} + \beta_k p^{(k)} \\ \tilde{p}^{(k+1)} &\leftarrow \tilde{r}^{(k+1)} + \beta_k \tilde{p}^{(k)} \end{aligned} \right. \end{aligned} \quad (7.9.2)$$

Comparing (7.9.2) with (7.7.24), we see that they are almost identical, except that (7.9.2) has extra recurrences to generate the dual vectors. If A is symmetric and $\tilde{r}^{(0)} = r^{(0)}$, then $\tilde{r}^{(k)} = r^{(k)}$ and $\tilde{p}^{(k)} = p^{(k)}$ for all k , and (7.9.2) collapses to (7.7.24).

The vectors generated by (7.9.2) satisfy several useful relationships, which are proved in Exercise 7.9.6. Generalizing Theorem 7.7.14 we have

$$\text{span}\{p^{(0)}, \dots, p^{(j)}\} = \text{span}\{r^{(0)}, \dots, r^{(j)}\} = \mathcal{K}_{j+1}(A, r^{(0)})$$

and the dual relationship

$$\text{span}\{\tilde{p}^{(0)}, \dots, \tilde{p}^{(j)}\} = \text{span}\{\tilde{r}^{(0)}, \dots, \tilde{r}^{(j)}\} = \mathcal{K}_{j+1}(A^T, \tilde{r}^{(0)})$$

for all j . In addition the $r^{(j)}$ sequence is biorthogonal to the $\tilde{r}^{(j)}$ sequence, that is, $\langle r^{(j)}, \tilde{r}^{(k)} \rangle = 0$ if $j \neq k$. Normally we have $\langle r^{(k)}, \tilde{r}^{(k)} \rangle \neq 0$. If not, the algorithm breaks down, since it then becomes impossible to compute β_k . Restating the biorthogonality conditions in terms of subspaces, we have the *Petrov-Galerkin* conditions

$$r^{(j)} \perp \mathcal{K}_j(A^T, \tilde{r}^{(0)}) \quad \text{and} \quad \tilde{r}^{(j)} \perp \mathcal{K}_j(A, r^{(0)}),$$

generalizing Corollary 7.7.19. The $p^{(j)}$ and $\tilde{p}^j$ sequences also satisfy a sort of biorthogonality condition, namely $\langle Ap^{(j)}, \tilde{p}^{(k)} \rangle = \langle p^{(j)}, A^T \tilde{p}^{(k)} \rangle = 0$ if $j \neq k$. Normally $\langle Ap^{(k)}, \tilde{p}^{(k)} \rangle \neq 0$. If not, the algorithm breaks down, since it is then impossible to compute α_k .

The convergence of BiCG is sometimes erratic. A number of improvements have been proposed. The algorithm QMR (Quasi Minimum Residual) [27, 28] uses essentially the BiCG recurrences to generate the Krylov subspaces but uses a criterion for choosing the approximate solution $x^{(j)}$ that is based on a quasi-minimization of the residual. Because the recurrences can sometimes break down, QMR is also often used with a “look ahead” feature that circumvents breakdowns.

A feature of BiCG and QMR that is sometimes a drawback is that each step requires a matrix-vector multiplication by A^T . In some applications the definition of A is so complex (a long subroutine) that it is not clear how to compute A^T . In such situations one needs an algorithm that does not use A^T . One such algorithm is BiCGSTAB (BiCG STABILized) [74], which replaces the A^T evaluation by a second A evaluation and also offers much smoother convergence. There is also a transpose-free version of QMR. Another popular algorithm of this type is called CGS (CG Squared) [65].

The algorithms BiCG, QMR, BiCGSTAB, and CGS are all based on short recurrences, so the memory and computation requirements per step do not increase as the algorithm proceeds, in contrast to GMRES.

All of the methods described in this section can be enhanced by the use of preconditioners. In the nonsymmetric case there is no need for a symmetric preconditioner. Incomplete LU decompositions are popular, but there is no end to the variety of preconditioners that have been tried.

Of the many iterative methods for solving nonsymmetric systems, there is no clear best choice.

MATLAB provides implementations of GMRES, BiCG, BiCGSTAB, CGS, and QMR (without “look ahead”). These are not the most efficient implementations, but they are convenient.

Exercise 7.9.3 Using the MATLAB m-file `condif.m` shown in Exercise 1.9.3, generate a matrix $A = \text{condif}(n, c)$. Take $n = 50$ (or larger) and $c = [0.2 \ 5]$. This 2500×2500 nonsymmetric sparse matrix is a discretized convection-diffusion operator. Try out some of MATLAB’s iterative solvers on this matrix. For example, try

```
n = 50; c = [.2 5]; A = condif(n,c);
nn = n^2; sol = randn(nn,1); % random solution vector
b = A*sol; % right-hand side vector
```

```
tol = 1e-12; maxit = 1000;
x = bicg(A,b,tol,maxit)
error = norm(x-sol)
```

Type `help bicg` for more information on how to use BiCG. After experimenting with BiCG, try QMR, BiCGSTAB, CGS, and GMRES. $\square$

Exercise 7.9.4 MATLAB provides a routine `luinc` that computes incomplete LU decompositions to use as preconditioners. Repeat Exercise 7.9.3 using a preconditioner. For example, try

```
droptol = 1e-1;
[M1,M2] = luinc(A,droptol);
spy(A)
spy(M1+M2)
x = bicg(A,b,tol,maxit,M1,M2)
error = norm(x-sol)
```

Try the various iterative methods. How does the preconditioner affect your results? Try a variety of drop tolerances. How does varying the drop tolerance affect the sparseness of $M1$ and $M2$ and the effectiveness of the preconditioner? $\square$

Exercise 7.9.5 Read more about MATLAB's iterative solvers. Figure out how to make them produce (residual) convergence histories. Then print out some convergence histories on semilog plots. For example, try

```
[x,flag,relres,iter,resvec1] = bicg(A,b,tol,maxit,M1,M2);
x1 = 1:size(resvec1,1);
semilogy(x1,resvec1,'b-'); % makes a blue solid line
```

Try comparing the convergence histories of BiCG, QMR, and GMRES, for example, in a single plot. $\square$

Exercise 7.9.6 Consider the BiCG algorithm (7.9.2) with the line $p^{(k+1)} \leftarrow r^{(k+1)} + \beta_k p^{(k)}$ replaced by an expression of the form (7.7.7) and the dual line $\tilde{p}^{(k+1)} \leftarrow \tilde{r}^{(k+1)} + \beta_k \tilde{p}^{(k)}$ replaced by an analogous expression with coefficients $\tilde{c}_{ki}$.

- (a) Suppose the biorthogonality conditions $\langle Ap^{(i)}, \tilde{p}^{(j)} \rangle = \langle p^{(i)}, A^T \tilde{p}^{(j)} \rangle = 0$ iff $i \neq j$ hold for $i \leq k$ and $j \leq k$. Show that $\langle Ap^{(k+1)}, \tilde{p}^{(j)} \rangle = 0$ and $\langle p^{(j)}, A^T \tilde{p}^{(k+1)} \rangle = 0$ for $j < k+1$ if and only if

$$c_{ki} = \frac{\langle Ar^{(k+1)}, \tilde{p}^{(i)} \rangle}{\langle Ap^{(i)}, \tilde{p}^{(i)} \rangle} \quad \text{and} \quad \tilde{c}_{ki} = \frac{\langle p^{(i)}, A^T \tilde{r}^{(k+1)} \rangle}{\langle p^{(i)}, A^T \tilde{p}^{(i)} \rangle}, \quad i = 0, \dots, k. \quad (7.9.7)$$

Thus we will use these values for c_{ki} and $\tilde{c}_{ki}$ from this point on, to ensure biorthogonality.

- (b) Extending Theorem 7.7.14, prove by induction on j that

$$\text{span}\{p^{(0)}, \dots, p^{(j)}\} = \text{span}\{r^{(0)}, \dots, r^{(j)}\} = \mathcal{K}_{j+1}(A, r^{(0)})$$

and

$$\text{span}\{\tilde{p}^{(0)}, \dots, \tilde{p}^{(j)}\} = \text{span}\{\tilde{r}^{(0)}, \dots, \tilde{r}^{(j)}\} = \mathcal{K}_{j+1}(A^T, \tilde{r}^{(0)})$$

so long as $\langle Ap^{(j)}, \tilde{p}^{(j)} \rangle \neq 0$.

- (c) Using the recurrences for $r^{(k+1)}$ and $\tilde{r}^{(k+1)}$ from (7.9.2), the biorthogonality from part (a), the subspace equalities from part (b), and the definition of α_k from (7.9.2), prove by induction that $\langle r^{(i)}, \tilde{r}^{(j)} \rangle = 0$ if $i \neq j$.
- (d) Using arguments like those in the proof of Theorem 7.7.20, show that $c_{ki} = 0$ and $\tilde{c}_{ki} = 0$ if $i < k$.
- (e) Show that $c_{kk} = \tilde{c}_{kk} = -\beta_k$, where β_k is as defined in (7.9.2). This completes the justification of (7.9.2).

□

This page intentionally left blank

Appendix A

Some Sources of Software for Matrix Computations

High-quality linear algebra software is available for free on the worldwide web. One of the most important sites is NETLIB:

<http://www.netlib.org/>

which offers numerical analysis software of all kinds. In particular we mention LAPACK [1], a large suite of Fortran routines for computations on dense matrices. There is also a C version called CLAPACK and a parallel version for distributed memory machines called ScaLAPACK [8]. The older packages LINPACK [18] and EISPACK [64] are also still available from NETLIB.

Another important site is the Guide to Available Mathematical Software (GAMS) at the National Institute of Standards and Technology (NIST):

<http://gams.nist.gov/>

which can help you locate mathematical software of all types.

MATLAB is proprietary software. If MATLAB is not already available to you, you can buy it from The MathWorks at

<http://www.mathworks.com/store/>

This page intentionally left blank

References

1. E. Anderson et al. *LAPACK Users' Guide*. SIAM, Philadelphia, Third edition, 1999. http://www.netlib.org/lapack/lug/lapack_lug.html.
2. W. E. Arnoldi. The principle of minimized iterations in the solution of the matrix eigenvalue problem. *Quart. Appl. Math.*, 9:17–29, 1951.
3. Z. Bai, J. Demmel, J. Dongarra, A. Ruhe, and H. van der Vorst, editors. *Templates for the Solution of Algebraic Eigenvalue Problems: A Practical Guide*. SIAM, Philadelphia, 2000.
4. Z. Bai, J. Demmel, and M. Gu. Inverse free parallel spectral divide and conquer algorithms for nonsymmetric eigenproblems. *Numer. Math.*, 76:279–308, 1997.
5. R. Barrett et al. *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*. SIAM, Philadelphia, 1994.
6. G. A. Birkhoff and R. E. Lynch. *Numerical Solution of Elliptic Problems*. SIAM Studies in Applied Mathematics. SIAM, Philadelphia, 1984.
7. A. Björck. *Numerical Methods for Least Squares Problems*. SIAM, Philadelphia, 1996.
8. L. S. Blackford et al. *ScaLAPACK Users' Guide*. SIAM, Philadelphia, 1997. www.netlib.org/scalapack/slug/scalapack_slug.html.
9. T. F. Chan and T. P. Mathew. Domain decomposition algorithms. *Acta Numerica*, pages 61–143, 1994.

10. P. Concus, G. H. Golub, and D. P. O'Leary. A generalize conjugate gradient method for the numerical solution of elliptic partial differential equations. In J. R. Bunch and D. J. Rose, editors, *Sparse Matrix Computations*, New York, 1976. Academic Press.
11. J. K. Cullum and R. A. Willoughby. *Lanczos Algorithms for Large Symmetric Eigenvalue Computations*. Birkhäuser, Boston, 1985.
12. J. J. M. Cuppen. A divide and conquer method for the symmetric tridiagonal eigenproblem., *Numer. Math.*, 36:177–195, 1981.
13. J. Demmel and B. Kågström. The generalized Schur decomposition of an arbitrary pencil $A - \lambda B$: robust software with error bounds and applications. *ACM Trans. Math. Software*, 19:160–201, 1993.
14. J. Demmel and K. Veselić. Jacobi's method is more accurate than QR. *SIAM J. Matrix Anal. Appl.*, 13:1204–1246, 1992.
15. J. W. Demmel. *Applied Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
16. I. S. Dhillon. *A New $O(n^2)$ Algorithm for the Symmetric Tridiagonal Eigenvalue/Eigenvector Problem*. PhD thesis, University of California, Berkeley, 1997.
17. I. S. Dhillon and B. N. Parlett. Orthogonal eigenvectors and relative gaps. LAPACK Working Note 154. Submitted to SIAM J. Matrix Anal. Appl., 2000.
18. J. J. Dongarra, J. R. Bunch, C. B. Moler, and G. W. Stewart. *LINPACK Users' Guide*. SIAM, Philadelphia, 1979.
19. J. J. Dongarra, F. G. Gustavson, and A. Karp. Implementing linear algebra algorithms for dense matrices on a vector pipeline machine. *SIAM Review*, 26:91–112, 1984.
20. J. J. Dongarra and D. C. Sorensen. A fully parallel algorithm for the symmetric eigenvalue problem. *SIAM J. Sci. Stat. Comput.*, 8:s139–s154, 1987.
21. I. S. Duff, A. M. Erisman, and J. K. Reid. *Direct Methods for Sparse Matrices*. Oxford University Press, 1986.
22. K. V. Fernando and B. N. Parlett. Accurate singular values and differential qd algorithms. *Numer. Math.*, 67:191–229, 1994.
23. G. E. Forsythe and P. Henrici. The cyclic Jacobi method for computing the principal values of a complex matrix. *Trans. Amer. Math. Soc.*, 94:1–23, 1960.
24. G. E. Forsythe and C. B. Moler. *Computer Solution of Linear Algebraic Systems*. Prentice-Hall, Englewood Cliffs, NJ, 1967.
25. J. G. F. Francis. The QR transformation, parts I and II. *Computer J.*, 4:265–272, 332–345, 1961.

26. R. W. Freund, G. H. Golub, and N. M. Nachtigal. Iterative solution of linear systems. *Acta Numerica*, 1:57–100, 1992.
27. R. W. Freund and N. M. Nachtigal. QMR: A quasi-minimal residual method for non-hermitian linear systems. *Numer. Math.*, 60:315–339, 1991.
28. R. W. Freund and N. M. Nachtigal. An implementation of the QMR method based on coupled two-term recurrences. *SIAM J. Sci. Comput.*, 15:313–337, 1994.
29. F. R. Gantmacher. *The Theory of Matrices*. Chelsea Publishing Co., New York, 1959.
30. A. George and J. W. Liu. *Computer Solution of Large Sparse Positive Definite Systems*. Prentice-Hall, Englewood Cliffs, NJ, 1981.
31. G. H. Golub. Numerical methods for solving linear least squares problems. *Numer. Math.*, 7:206–216, 1965.
32. G. H. Golub and W. Kahan. Calculating the singular values and pseudo-inverse of a matrix. *SIAM J. Numer. Anal.*, 2:202–224, 1965.
33. G. H. Golub and C. F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, Baltimore, Third edition, 1996.
34. A. Greenbaum. *Iterative Methods for Solving Linear Systems*. SIAM, Philadelphia, 1997.
35. M. H. Gutknecht. Lanczos-type solvers for nonsymmetric linear systems of equations. *Acta Numerica*, 6:271–397, 1997.
36. W. Hackbusch. *Iterative Solution of Large, Sparse Systems of Equations*, volume 95 of *Applied Mathematical Sciences*. Springer-Verlag, 1994.
37. C. R. Hadlock. *Field Theory and Its Classical Problems*. The Carus Mathematical Monographs. Mathematical Association of America, 1978.
38. L. A. Hageman and D. M. Young. *Applied Iterative Methods*. Academic Press, New York, 1981.
39. M. R. Hestenes and E. Stiefel. Methods of conjugate gradients for solving linear systems. *J. Res. Nat. Bur. Standards*, 49:409–436, 1952.
40. D. J. Higham and N. J. Higham. *MATLAB Guide*. SIAM, Philadelphia, 2000.
41. N. J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, Philadelphia, 1996.
42. R. A. Horn and C. A. Johnson. *Matrix Analysis*. Cambridge University Press, 1985.

43. A. S. Householder. *The Theory of Matrices in Numerical Analysis*. Blaisdell, New York, 1964. Reprinted by Dover, New York, 1975.
44. C. G. J. Jacobi. Über ein leichtes Verfahren die in der Theorie der Säculärstörungen vorkommenden Gleichungen numerisch aufzulösen. *J. Reine Angew. Math.*, 30:51–94, 1846.
45. V. N. Kublanovskaya. On some algorithms for the solution of the complete eigenvalue problem. *USSR Comput. Math. and Math. Phys.*, 3:637–657, 1961.
46. P. Lancaster and M. Tismenetsky. *The Theory of Matrices*. Academic Press, Second edition, 1985.
47. C. Lanczos. An iteration method for the solution of the eigenvalue problem of linear differential and integral operators. *J. Res. Nat. Bur. Stand.*, 45:255–282, 1950.
48. C. L. Lawson and R. J. Hanson. *Solving Least Squares Problems*. Prentice-Hall, Englewood Cliffs, NJ, 1974.
49. R. B. Lehoucq, D. C. Sorensen, and C. Yang. *ARPACK Users' Guide: Solution of Large-Scale Eigenvalue Problems with Implicitly Restarted Arnoldi Methods*. SIAM, Philadelphia, 1998. <http://www.caam.rice.edu/software/ARPACK/index.html>.
50. G. S. Minimis and C. C. Paige. Implicit shifting in the QR and related algorithms. *SIAM J. Matrix Anal. Appl.*, 12:385–400, 1991.
51. C. B. Moler and G. W. Stewart. An algorithm for generalized matrix eigenvalue problems. *SIAM J. Numer. Anal.*, 10:241–256, 1973.
52. M. L. Overton. *Numerical Computing with IEEE Floating Point Arithmetic*. SIAM, Philadelphia, 2001.
53. C. Paige and M. Saunders. Solution of sparse indefinite systems of linear equations. *SIAM J. Numer. Anal.*, 12:617–629, 1975.
54. B. N. Parlett. *The Symmetric Eigenvalue Problem*. Prentice-Hall, Englewood Cliffs, NJ, 1980. Reprinted by SIAM, Philadelphia, 1997.
55. B. N. Parlett. The new qd algorithm. *Acta Numerica*, 4:459–491, 1995.
56. B. N. Parlett and I. S. Dhillon. Relatively robust representations of symmetric tridiagonals. *Linear Algebra Appl.*, 309:121–151, 2000.
57. H. Rutishauser. Der Quotienten-Differenzen-Algorithmus. *Z. angew. Math. Physik*, 5:233–251, 1954.
58. H. Rutishauser. *Der Quotienten-Differenzen-Algorithmus*. Number 7 in Mitt. Inst. angew. Math. ETH. Birkhäuser, Basel, 1957.

59. H. Rutishauser. Solution of eigenvalue problems with the LR-transformation. *Nat. Bur. Standards Appl. Math. Series*, 49:47–81, 1958.
60. Y. Saad and M. Schultz. GMRES: a generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM J. Sci. Statist. Comput.*, 7:856–869, 1986.
61. R. D. Skeel. Iterative refinement implies numerical stability for Gaussian elimination. *Math. Comp.*, 35:817–832, 1980.
62. G. L. G. Sleijpen and H. A. van der Vorst. A Jacobi-Davidson iteration method for linear eigenvalue problems. *SIAM J. Matrix Anal. Appl.*, 17:401–425, 1996.
63. B. F. Smith, P. E. Bjørstad, and W. D. Gropp. *Domain Decomposition: Parallel Multilevel Methods for Elliptic Partial Differential Equations*. Cambridge University Press, 1996.
64. B. T. Smith et al. *Matrix Eigensystem Routines — EISPACK Guide*. Springer-Verlag, New York, Second edition, 1976.
65. P. Sonneveld. CGS, a fast Lanczos-type solver for nonsymmetric linear systems. *SIAM J. Sci. Statist. Comput.*, 10:36–52, 1989.
66. G. W. Stewart. Error and perturbation bounds for subspaces associated with certain eigenvalue problems. *SIAM Review*, 15:727–764, 1973.
67. G. W. Stewart. *Introduction to Matrix Computations*. Academic Press, New York, 1973.
68. V. Strassen. Gaussian elimination is not optimal. *Numer. Math*, 13:354–356, 1969.
69. J. C. Strikwerda. *Finite Difference Schemes and Partial Differential Equations*. Chapman and Hall, New York, 1989.
70. A. E. Taylor and D. C. Lay. *Introduction to Functional Analysis*. Krieger, Malabar, FL, Second edition, 1986.
71. L. N. Trefethen and D. Bau, III. *Numerical Linear Algebra*. SIAM, Philadelphia, 1997.
72. A. M. Turing. Rounding-off errors in matrix processes. *Quart. J. Appl. Math.*, 1:287–308, 1948.
73. E. E. Tyrtysnikov. *A Brief Introduction to Numerical Analysis*. Birkhäuser, Boston, 1997.
74. H. A. van der Vorst. Bi-CGSTAB: A fast and smoothly converging variant of Bi-CG for the solution of nonsymmetric linear systems. *SIAM J. Sci. Statist. Comput.*, 13:631–644, 1992.

75. P. Van Dooren. The computation of Kronecker's canonical form of a singular pencil. *Linear Algebra Appl.*, 27:103–141, 1979.
76. R. S. Varga. *Matrix Iterative Analysis*. Prentice-Hall, Englewood Cliffs, NJ, 1962.
77. D. S. Watkins. *Fundamentals of Matrix Computations*. John Wiley and Sons, New York, First edition, 1991.
78. D. S. Watkins. Performance of the QZ algorithm in the presence of infinite eigenvalues. *SIAM J. Matrix Anal. Appl.*, 22:364–375, 2000.
79. D. S. Watkins and L. Elsner. Convergence of algorithms of decomposition type for the eigenvalue problem. *Linear Algebra Appl.*, 143:19–47, 1991.
80. D. S. Watkins and L. Elsner. Theory of decomposition and bulge-chasing algorithms for the generalized eigenvalue problem. *SIAM J. Matrix Anal. Appl.*, 15:943–967, 1994.
81. J. H. Wilkinson. *The Algebraic Eigenvalue Problem*. Clarendon Press, Oxford University, 1965.
82. J. H. Wilkinson. Kronecker's canonical form and the QZ algorithm. *Linear Algebra Appl.*, 28:285–303, 1979.
83. J. H. Wilkinson and C. Reinsch, editors. *Handbook for Automatic Computation, Volume II, Linear Algebra*. Springer-Verlag, New York, 1971.
84. D. M. Young. *Iterative Solution of Large Linear Systems*. Academic Press, New York, 1971.

Index

- Abel, N. H., 310, 478
- Absolute value of a matrix, 159, 176
- Algebraic multiplicity, 306, 417
- Ammar-Dillo conjecture, 604
- Angle between two vectors, 186
- Arnoldi process, 435
 - implicitly restarted (IRA), 451
- Arrowhead matrix, 63
- Augmented matrix, 71
- Back substitution, 29
- Backward error analysis, 145
- Backward stability, 146
 - and residual, 146, 463
 - componentwise, 161, 179
 - normwise, 161
 - of Cholesky's method, 164
 - of forward substitution, 159
 - of Gaussian elimination, 162
 - of matrix multiplication, 211
 - of rotators and reflectors, 206, 211
- Band width, 54
- Banded matrix, 54
- Basis (of a subspace), 224
- Bauer's Theorem, 173
- Bauer-Fike Theorem, 463
- Best approximation theorem, 243, 583
- BiCGSTAB, 599
- Biconjugate-gradient (BiCG) algorithm, 598
- Bidiagonal matrix, 402
 - proper, 405
- Biorthogonal vectors, 465, 599
- Bisection method, 483
- BLAS, 102
- Block Gaussian elimination, 90
- Block iteration, 541
- Bucky ball, 64
- Bulge-chasing algorithm, 375, 380, 407, 486, 513
- Bunch-Kaufman algorithm, 106
- Cache, 10
- Cancellation, 144
 - in Gaussian elimination, 151
- Catastrophic cancellation, 144
- Cauchy-Schwarz inequality, 113, 186
- CGS, 599
- Characteristic equation, 305, 505
- Characteristic polynomial, 305, 505
- Chebyshev acceleration, 550
- Chebyshev polynomials, 448
 - and convergence of CG algorithm, 594
 - and convergence of Lanczos process, 450
- Cholesky decomposition, 34
 - downdating, 253, 255, 257
 - updating, 251
- Cholesky's method
 - backward stability, 164
 - block, 45
 - bordered form, 43
 - inner-product form, 36
 - outer-product form, 42
- Cofactor method, 103

- Column scaling, 171
- Column space, 242
- Companion matrix, 309
- Complete pivoting, 104
- Componentwise backward stability, 179
- Condition number, 121
 - control of, 284
 - estimation of, 131
 - geometric view, 123, 128
 - of nonsquare matrix, 232
 - of simple eigenvalue, 466
 - of simple eigenvector, 471
 - of square matrix, 121
 - Skeel, 177
- Conductance, 50
- Conformable partition, 9
- Congruence, 483
 - of matrix pairs, 509
- Conjugate (vectors), 583
- Conjugate transpose, 49
- Conjugate-gradient (CG) method, 576
 - and Lanczos process, 588, 590, 597
 - as acceleration technique, 550
 - convergence of, 590
 - derivation, 581
 - preconditioned, 578
- Contraction number, 315
- Convection-diffusion operator, 107, 109
- Convergence
 - cubic, 328
 - linear, 315
 - quadratic, 328
 - slower than linear, 315
- Convergence rate, asymptotic ($R_\infty(G)$), 548
 - of classical methods, 549
- Convergence ratio, 315
- Cramer's rule, 105
- Crout reduction, 81
- Cubic convergence, 328
- Cuppen's algorithm, 491
 - secular equation, 494
- Curse of dimensionality, 529
- Cuthill-McKee ordering, reverse, 64
- Damped iteration, 553
- Damped Jacobi method, 544
- Damped pendulum, 299
- Davidson's method, 458
- Decomposition
 - Cholesky, 34
 - LDL^T , 84, 479
 - LDV , 84
 - LR , 482
 - LU , 80
 - $MD^{-1}M^T$, 85
 - QR , 190
 - Schur, 337
 - singular value, 262
 - spectral, 339
 - twisted, 489
- Defective matrix, 308
- Deflation in the QR algorithm, 361
- Departure from normality, 468
- Descent method, 560
 - CG, 576
 - Gauss-Seidel, 561
 - preconditioned CG, 578
 - SOR, 562
 - steepest, 562
- Determinant, computation of, 104
- Diagonal matrix, 84
- Diagonalization theorem, 335
- Differential equations, 289, 502
 - asymptotically stable, 298
 - homogeneous linear, 291
 - nonhomogeneous linear, 294
 - nonlinear, 298
 - unstable, 298
- Dimension (of a subspace), 224
- Direct sum (of subspaces), 241
- Direct vs. iterative methods, 310
- Distance to nearest
 - rank-deficient matrix, 270
 - singular matrix, 274
- Divide-and-conquer algorithm, 491
- Domain decomposition, 549
- Dominant eigenvalue, 314
- Doolittle reduction, 81
- Downdating QR
 - deleting a column, 254
 - deleting a row, 252
- Dqds algorithm, 487
- Duality in subspace iteration, 428
- Eigenpair, 305
- Eigenspace, 414
- Eigenvalue, 305, 505
 - condition number, 466
- Eigenvalues of normal matrices, 345
- Eigenvector, 305, 505
 - condition number, 471
 - left, 331, 465
 - right, 465
- Elementary row operation, 71
- Elimination
 - Gauss-Jordan, 105
 - Gaussian, 71
- Energy inner product and norm, 582
- Envelope of a matrix, 57
- Equivalence
 - of linear systems, 70
 - of matrices, 262
 - of matrix pairs (pencils), 507
 - orthogonal, 262, 401

- Error analysis
 - backward vs. forward, 145
 - componentwise, 175
 - a priori* vs. *a posteriori*, 137
- Error, absolute vs. relative, 141
- Euclidean norm, 112
- Exact shift, 323, 367
- Exceptional shift, 367
- Fast Poisson solver, 529
- Fast rotators, 194
- Fill-in, 63
- Finite difference method, 18, 522, 524
- Five-point stencil, 525
- Floating-point
 - arithmetic, 139
 - IEEE standard, 140
- Flop, 3
- Fortran-90, 101
- Forward substitution, 24
 - backward stability, 159
 - block, 30
 - column oriented, 27
 - row oriented, 24
- Frobenius norm, 115
- Full rank, 213
- Fundamental subspaces, 241, 264
- Galerkin condition, 447, 598
 - Petrov, 599
- Gauss transform, 89, 210
- Gauss-Jordan elimination, 105
- Gauss-Seidel method, 533
 - as a descent method, 561
 - as smoother for multigrid, 544
 - block, 541
 - convergence rate, 549
 - line, 542
 - orderings, 536
 - red-black, 537
 - symmetric, 538
- Gaussian elimination
 - backward stability, 162
 - block, 90, 102
 - column oriented, 100
 - generic, 91
 - inner-product formulation, 81
 - row oriented, 100
 - sparse, 106
 - with pivoting, 93
 - with small pivot, 154
 - without pivoting, 71
- Generalized eigenvalue problem, 502, 505
- Generalized inverse, Moore-Penrose, 277
- Geometric multiplicity, 342, 417
- Gerschgorin disk, 473
- Givens rotator, 192
- GMRES, 597
- Gradient, 560
- Gram-Schmidt process
 - and QR decomposition, 227
 - classical, 226–227
 - modified, 229
 - reorthogonalization, 233
- Harmonic Ritz value, 461
- Hermitian matrix, 49, 337
- Hessenberg matrix, 329, 349
 - j -Hessenberg, 460
 - 2-Hessenberg, 379
 - proper (unreduced), 365
- Hessenberg-triangular form, 510
- Hilbert matrix, 123, 150, 248
- Householder transformation, 196
- Hyperbolic transformation, 256
- Hyperplane, 196
- Ideal computer, 142
- Idempotent, 195
- Identity matrix, 12
- IEEE floating-point standard, 140
- Ill-conditioned matrix, 121
- Implicit- Q theorem, 381, 408, 514
- Implicitly restarted Arnoldi (IRA), 451
- Inductance, 290
- Induction, mathematical, 31
- Inertia, 483
- Inner product, 186, 581–582
 - complex, 206
 - energy, 582
 - induced by H , 582
 - of functions, 247
- Invariant subspace, 414
- Inverse matrix, 12
 - computation of, 103
 - of a sparse matrix, 59
 - pseudoinverse, 277
- Inverse power method (inverse iteration), 319
- Invertible matrix, 13
- IRA, 451
- Isometric matrix (isometry), 221
- Iterative refinement, 178
- Iterative vs. direct methods, 310
- Jacobi rotator, 477, 495
- Jacobi's method
 - damped, 544
 - for eigenvalue problem, 477
 - for linear system, 531
 - block variant, 541
 - convergence rate, 549
- Jacobi-Davidson method, 458
 - inexact solves, 459
 - purging, 459
- Jacobian matrix, 299
- Jordan block, 309, 554
- Jordan canonical form, 335, 554

- Kahan matrix, 149, 273
 - Kernel, 241
 - Kirchhoff's laws
 - current, 13
 - voltage, 15
 - Kronecker product, 556
 - Krylov matrix, 390, 446
 - Krylov subspace methods, 596
 - Krylov subspace, 436, 585
 - Lanczos process, 441
 - connection with CG, 588, 590, 597
 - LDL^T decomposition, 84
 - LDV decomposition, 84
 - Least squares problem, 181
 - and SVD, 275
 - continuous, 247
 - discrete, 182, 243
 - solution in full-rank case, 213
 - solution in rank-deficient case, 216, 275
 - Line iteration, 542
 - Line search, 560
 - exact vs. inexact, 561
 - Linear combination, 223
 - Linear convergence, 315
 - Linear independence and dependence, 224
 - Lotkin matrix, 153
 - LR algorithm, 485
 - LU decomposition, 80
 - Manhattan metric, 114
 - Markov chain, 332
 - Matrix norm, 115
 - Matrix pencil (pair), 505
 - equivalence, 507
 - symmetric, 508
 - Maxmag, 123
 - $MD^{-1}M^T$ decomposition, 85
 - Minimum-degree ordering
 - column, 108
 - symmetric, 64
 - Minmag, 123
 - MINRES, 597
 - Monic polynomial, 309
 - Moore-Penrose generalized inverse, 277
 - Multicolor ordering, 537
 - Multigrid, 544, 549
 - damped Jacobi smoother, 544
 - Gauss-Seidel smoother, 544
 - Multiplication
 - matrix, 6
 - matrix-vector, 2
 - Multiplicity
 - algebraic vs. geometric, 417
 - algebraic, 306
 - geometric, 342
 - NASA Airfoil, 69
 - Nested-dissection ordering, 69
 - Newton's second law, 297
 - Nondefective matrix, 308
 - Nonsingular matrix, 13
 - Norm, 112
 - energy, 582
 - matrix, 115
 - p -norm, 118
 - ∞ -norm (row sum), 119
 - 1-norm (column sum), 119
 - Frobenius, 115
 - induced, 116
 - spectral (2-norm), 118
 - of a function, 247
 - vector, 112
 - A -norm, 115
 - p -norm, 114
 - ∞ -norm, 114
 - 1-norm, 114
 - Euclidean (2-norm), 112
 - taxicab (1-norm), 114
- Normal equations, 244
- Normal matrix, 339
- Null space, 241
- Ohm's law, 13
- Operator norm, 116
- Orthogonal complement, 240
- Orthogonal decomposition, 273
- Orthogonal equivalence, 262, 401
- Orthogonal matrix, 187
- Orthogonal projection (of vector onto subspace), 240
- Orthogonality
 - of functions, 247
 - of vectors, 186
- Orthonormal vectors, 221
- Orthoprojector, 195
- Overdetermined system, 184
- Overflow, 140
- Partial pivoting, 94
- Partition, conformable, 9
- Pencil (pair), 505
 - equivalence, 507
 - regular vs. singular, 506
 - symmetric, 508
- Permutation matrix, 97, 187
- Perron-Frobenius theory, 331
- Petrov-Galerkin conditions, 599
- Pivot, 94
- Pivoting
 - complete, 104
 - partial, 94
- Plane rotator, 192
- Poisson's equation, 524
- Positive definite matrix, 33
 - complex, 49
- Positive semidefinite matrix, 344

- Power method, 314
 - convergence rate, 315
 - inverse, 319
- Preconditioner, 571
 - and Richardson's method, 553
 - as splitting matrix, 544
 - diagonal ($\approx$ Jacobi), 573
 - domain decomposition, 575
 - incomplete factorization, 575, 581
 - multigrid, 575
 - SSOR, 574
- Principal submatrix, 43, 72
- Projector, 195
- Proper Hessenberg matrix, 365
- Pseudoinverse, 277
- Purging, 459
- Pythagorean theorem, 243
- QMR, 599
- QR algorithm, 356
 - accelerating convergence by shifting, 361
 - as Rayleigh quotient iteration, 430
 - as simultaneous iteration, 426
 - deflation, 361
 - exact shift, 367
 - exceptional shift, 367
 - implicit vs. explicit, 373
 - implicit
 - double step, 377
 - single step, 372
 - on Hessenberg matrices, 358
 - Rayleigh quotient shift, 362
 - reduction, 365
 - singular case, 367
 - Wilkinson shift, 364
 - with symmetric matrix, 377
- QR decomposition, 190
 - and Gram-Schmidt process, 227
 - by reflectors, 203
 - by rotators, 193
 - complex, 208
 - condensed, 222
 - nonsquare, 213
 - rank-deficient case, 217
 - uniqueness, 204, 223
 - with column pivoting, 217
- QR iteration (QR step), 357
- Quadratic convergence, 328
- Quasi-triangular matrix, 341
- Quotient-difference (qd) algorithm, 485
 - differential form, 487
- QZ algorithm, 511
- Range, 241
- Rank, 213, 241
- Rank-one update
 - eigenvalue problem, 492
 - of identity matrix, 90, 209
- Rayleigh quotient iteration, 326
 - convergence rate, 327
 - cubic convergence, 328
 - in QR algorithm, 430
 - quadratic convergence, 328
- Rayleigh quotient, 325
 - matrix, 447
 - of normal matrix, 345
 - shift, 362
- Rayleigh-Ritz-Galerkin condition, 447
- Red-black ordering, 537
- Refinement, iterative, 178
- Reflector, 196
 - backward stability, 206, 211
 - complex, 207
- Regular pencil (pair), 506
- Relatively robust representation (RRR), 480
- Reorthogonalization, 233
- Residual, 137
 - and backward stability, 146, 463
- Richardson's method, 545
- Ritz pair, 439
- Ritz value, 439, 447
 - harmonic, 461
- Ritz vector, 439, 447
- Rotator, 188
 - backward stability, 206, 211
 - complex, 207
 - fast, 194
 - Givens, 192
 - Jacobi, 477, 495
 - plane, 192
- Roundoff error, 140
- Roundoff, unit, 142
- Row operation, elementary, 71
- Row scaling, 171
- Row space, 242
- RRR algorithm, 488
- Scaling, 171
 - bad, 127
- Schur complement, 48, 90–91, 258
- Schur's theorem, 337
 - generalized, 509, 517
 - real (Wintner-Murnaghan), 341
- Schwarz inequality, 113, 186
- Scientific notation, 139
- Secular equation, 494
- Semiband width, 54
- Semisimple matrix, 308
- Shift, 319
 - exact, 323, 367
 - exceptional, 367
 - Rayleigh quotient, 362
 - ultimate, 393
 - Wilkinson, 364
- Shift-and-invert strategy, 319

- and Arnoldi process, 441
- and IRA, 455
- and subspace iteration, 434
- Similar matrices, 334
- Similarity, 334
 - orthogonal and unitary, 336
- Simultaneous iteration, 423
 - on sparse matrices, 433
- Singular matrix, 13
- Singular pencil (pair), 506
- Singular value decomposition (SVD), 262
- Singular vectors, 262
- Skeel condition number, 177
- Skew-Hermitian matrix, 339
- Skew-symmetric matrix, 347
- Slicing method, 483
- SOR method, 539
 - as a descent method, 562
 - block, 541
 - convergence rate, 549
 - line, 542
 - red-black, 542
 - symmetric, 539
- Span of a set of vectors, 223
- Spanning set (of a subspace), 224
- Spectral norm, 118
- Spectral radius, 547
- Spectral theorem
 - for Hermitian matrices, 339
 - for normal matrices, 340
 - for real symmetric matrices, 340
- Spectrum, 305
- Splitting matrix (= preconditioner), 544
- Spy plot, 65
- SSOR, 539
- Stability, backward, 146
- Standard basis, 221
- Steepest descent method, 562
 - geometric interpretation, 565
 - preconditioned, 572
- Stiffness matrix, 17, 560
- Stochastic matrix, 331–332
- Strain energy, 33, 51, 560
- Strassen's method, 11
- Submatrix, principal, 72
- Subspace iteration, 420
 - duality in, 428
- Subspace, 223
 - eigenspace, 414
 - invariant, 414
- Substitution
 - back, 29
 - forward, 24
- Successive overrelaxation (SOR), 539
 - block, 541
 - line, 542
 - red-black, 542
 - symmetric, 539
- Sum (of subspaces), 240
- SVD, 262
 - and spectral decomposition, 346
 - computation of, 399
 - condensed, 264
 - diagram, 263
 - geometric, 398
 - revisited, 396
- Swamping, 155, 172
- Sylvester's law of inertia, 483
 - proof, 498
- Symmetric Lanczos process, 441
- Symmetric matrix, 32
- Symmetric SOR, 539
- SYMMLQ, 598
- Systems of differential equations, 289, 502
 - asymptotically stable, 298
 - homogeneous linear, 291
 - nonhomogeneous linear, 294
 - nonlinear, 298
 - unstable, 298
- Taxicab norm, 114
- Tensor product, 556
- Trace, 170, 347
- Transpose, 32
- Transpose, conjugate, 49
- Triangle inequality, 112
- Triangular matrix, 23
- Tridiagonal matrix, 349
- Twisted factorization, 488–489
- Ultimate shift strategy, 393
- Underflow, 140
 - gradual, 148
 - harmful, 199
- Unit lower-triangular matrix, 78
- Unit roundoff, 142
- Unitary matrix, 206, 336
- Unreduced Hessenberg matrix, 365
- Updating the QR decomposition
 - adding a column, 254
 - adding a row, 249
- Vandermonde matrix, 232
- Vector norm, 112
- Well-conditioned matrix, 121
- Wilkinson shift, 364
- Wintner-Murnaghan theorem, 341
 - proof, 346, 418

Index of MATLAB Terms

acosh, 450
airfoil, 69
backslash, 15
bicg, 600
bicgstab, 600
bucky, 64
cgs, 600
chol, 39
cholinc, 581
computer, 148
cond, 122
condeig, 468
condest, 132
cosh, 450
cputime, 5
delsq, 68
demo, 5
det, 104
diag, 220
diary, 5
eig, 300, 504
eigs, 452
elmat, 70
eps, 169
eye, 170
format, 153
 long e, 500
 long, 153
 short e, 272
full, 322
gallery, 70
gmres, 600
help, 5
hess, 355
hilb, 123
inf, 148
inv, 59, 104
isieee, 148
issparse, 67
kahan, 273
kron, 109
long, 153
lotkin, 153
lu, 98
luinc, 600
m-file, 5
matrix-matrix multiplication, 7
matrix-vector multiplication, 5
more, 53
nan, 148
nnz, 67
norm, 273
numgrid, 68
ones, 153
pcg, 580
pinv, 280
plot, 184
poly, 304

- print, 300
- qmr, 600
- qr, 205, 220
- qrdelete, 255
- qrinsert, 254
- rand, 332
- randn, 5
- randperm, 68
- rank, 270
- roots, 304
- rsf2csf, 349
- save, 54
- schur, 349
- semicolon, 6
- semilogy, 600
- size, 67
- sort, 445
- sparfun, 64
- speye, 64
- spy, 65
- svd, 265
- symmmd, 68
- symrcm, 68
- tic, 68
- title, 300
- toc, 68
- toeplitz, 59
- transpose, 98
 - conjugate, 208
- wathen, 70
- west0479, 67
- xlabel, 300
- ylabel, 300

PURE AND APPLIED MATHEMATICS

A Wiley-Interscience Series of Texts, Monographs, and Tracts

Founded by RICHARD COURANT

Editors: MYRON B. ALLEN III, DAVID A. COX, PETER LAX

Editors Emeriti: PETER HILTON, HARRY HOCHSTADT, JOHN TOLAND

- ADÁMEK, HERRLICH, and STRECKER—Abstract and Concrete Categories
ADAMOWICZ and ZBIERSKI—Logic of Mathematics
AINSWORTH and ODEN—A Posteriori Error Estimation in Finite Element Analysis
AKIVIS and GOLDBERG—Conformal Differential Geometry and Its Generalizations
ALLEN and ISAACSON—Numerical Analysis for Applied Science
*ARTIN—Geometric Algebra
AUBIN—Applied Functional Analysis, Second Edition
AZIZOV and IOKHVIDOV—Linear Operators in Spaces with an Indefinite Metric
BERG—The Fourier-Analytic Proof of Quadratic Reciprocity
BERMAN, NEUMANN, and STERN—Nonnegative Matrices in Dynamic Systems
BERKOVITZ—Convexity and Optimization in $\mathbb{R}^n$
BOYARINTSEV—Methods of Solving Singular Systems of Ordinary Differential Equations
BURK—Lebesgue Measure and Integration: An Introduction
*CARTER—Finite Groups of Lie Type
CASTILLO, COBO, JUBETE, and PRUNEDA—Orthogonal Sets and Polar Methods in Linear Algebra: Applications to Matrix Calculations, Systems of Equations, Inequalities, and Linear Programming
CASTILLO, CONEJO, PEDREGAL, GARCÍA, and ALGUACIL—Building and Solving Mathematical Programming Models in Engineering and Science
CHATELIN—Eigenvalues of Matrices
CLARK—Mathematical Bioeconomics: The Optimal Management of Renewable Resources, Second Edition
COX—Primes of the Form $x^2 + ny^2$: Fermat, Class Field Theory, and Complex Multiplication
*CURTIS and REINER—Representation Theory of Finite Groups and Associative Algebras
*CURTIS and REINER—Methods of Representation Theory: With Applications to Finite Groups and Orders, Volume I
CURTIS and REINER—Methods of Representation Theory: With Applications to Finite Groups and Orders, Volume II
DINCULEANU—Vector Integration and Stochastic Integration in Banach Spaces
*DUNFORD and SCHWARTZ—Linear Operators
 Part 1—General Theory
 Part 2—Spectral Theory, Self Adjoint Operators in Hilbert Space
 Part 3—Spectral Operators
FARINA and RINALDI—Positive Linear Systems: Theory and Applications
FOLLAND—Real Analysis: Modern Techniques and Their Applications
FRÖLICHER and KRIEGL—Linear Spaces and Differentiation Theory
GARDINER—Teichmüller Theory and Quadratic Differentials
GREENE and KRANTZ—Function Theory of One Complex Variable
*GRIFFITHS and HARRIS—Principles of Algebraic Geometry

*Now available in a lower priced paperback edition in the Wiley Classics Library.

†Now available in paperback.

- GRILLET—Algebra
- GROVE—Groups and Characters
- GUSTAFSSON, KREISS and OLIGER—Time Dependent Problems and Difference Methods
- HANNA and ROWLAND—Fourier Series, Transforms, and Boundary Value Problems, Second Edition
- *HENRICI—Applied and Computational Complex Analysis
 Volume 1, Power Series—Integration—Conformal Mapping—Location of Zeros
 Volume 2, Special Functions—Integral Transforms—Asymptotics—Continued Fractions
 Volume 3, Discrete Fourier Analysis, Cauchy Integrals, Construction of Conformal Maps, Univalent Functions
- *HILTON and WU—A Course in Modern Algebra
- *HOCHSTADT—Integral Equations
- JOST—Two-Dimensional Geometric Variational Procedures
- KHAMSI and KIRK—An Introduction to Metric Spaces and Fixed Point Theory
- *KOBAYASHI and NOMIZU—Foundations of Differential Geometry, Volume I
- *KOBAYASHI and NOMIZU—Foundations of Differential Geometry, Volume II
- KOSHY—Fibonacci and Lucas Numbers with Applications
- LAX—Linear Algebra
- LOGAN—An Introduction to Nonlinear Partial Differential Equations
- McCONNELL and ROBSON—Noncommutative Noetherian Rings
- MORRISON—Functional Analysis: An Introduction to Banach Space Theory
- NAYFEH—Perturbation Methods
- NAYFEH and MOOK—Nonlinear Oscillations
- PANDEY—The Hilbert Transform of Schwartz Distributions and Applications
- PETKOV—Geometry of Reflecting Rays and Inverse Spectral Problems
- *PRENTER—Splines and Variational Methods
- RAO—Measure Theory and Integration
- RASSIAS and SIMSA—Finite Sums Decompositions in Mathematical Analysis
- RENELT—Elliptic Systems and Quasiconformal Mappings
- RIVLIN—Chebyshev Polynomials: From Approximation Theory to Algebra and Number Theory, Second Edition
- ROCKAFELLAR—Network Flows and Monotropic Optimization
- ROITMAN—Introduction to Modern Set Theory
- *RUDIN—Fourier Analysis on Groups
- SENDOV—The Averaged Moduli of Smoothness: Applications in Numerical Methods and Approximations
- SENDOV and POPOV—The Averaged Moduli of Smoothness
- *SIEGEL—Topics in Complex Function Theory
 Volume 1—Elliptic Functions and Uniformization Theory
 Volume 2—Automorphic Functions and Abelian Integrals
 Volume 3—Abelian Functions and Modular Functions of Several Variables
- SMITH and ROMANOWSKA—Post-Modern Algebra
- STAKGOLD—Green's Functions and Boundary Value Problems, Second Edition
- *STOKER—Differential Geometry
- *STOKER—Nonlinear Vibrations in Mechanical and Electrical Systems
- *STOKER—Water Waves: The Mathematical Theory with Applications
- WATKINS—Fundamentals of Matrix Computations, Second Edition
- WESSELING—An Introduction to Multigrid Methods
- †WHITHAM—Linear and Nonlinear Waves
- †ZAUDERER—Partial Differential Equations of Applied Mathematics, Second Edition

*Now available in a lower priced paperback edition in the Wiley Classics Library.

†Now available in paperback.